

JAVA GENERICS

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



- I tipi generici, **Generics**, sono stati aggiunti in Java dalla versione 5.0
- Aggiungono un ulteriore livello di astrazione sui tipi di dato
- Utile per definire classi che possono operare su tipi di dato differente
 - Ad esempio, possiamo implementare strutture dati di tipo dizionario senza fissare a priori il tipo della chiave e dei dati
- Possiamo quindi parametrizzare i metodi rispetto ad un tipo generico
- La definizione di una classe generica si esprime con la sintassi

NomeDellaClasse $\langle T \rangle$

T è un tipo generico utilizzabile nella classe e nei suoi metodi

- Anche le interfacce possono essere *generiche* (vedi [Comparable](#))

ESEMPIO DI CLASSE GENERICA

```
1 public class Pair<T> {  
2     private T first;  
3     private T second;  
4  
5     public Pair(T first, T second) {  
6         this.first = first;  
7         this.second = second;  
8     }  
9     public T getFirst() {return first;}  
10    public T getSecond() {return second;}  
11  
12    public void swap() {  
13        T tmp = this.first;  
14        this.first = this.second;  
15        this.second = tmp;  
16    }  
17    public String toString() {  
18        return "[" + first + "," + second + "]";  
19    }  
20 }
```

ESEMPIO DI UTILIZZO DI CLASSE GENERICA

```
1 public static void main(String args[]) {  
2     Pair<Integer> I = new Pair<Integer>(1,2);  
3     Pair<String> S = new Pair<>("Mario","Maria");  
4  
5     System.out.println(I);  
6     I.swap();  
7     System.out.println(I);  
8 }
```

- Possiamo istanziare *Pair* con coppie di interi, stringhe, ecc
- E' necessario specificare un tipo effettivo, non possiamo istanziare con

```
Pair<T> I = new Pair<T>(1,2);
```

- Non è necessario esplicitare il tipo quando chiamiamo il costruttore
 - Nell'esempio (riga 3), il tipo *String* è inferito dalla dichiarazione

TIPI GENERICI MULTIPLI

- Una classe generica può avere più di un tipo generico

```
1 public class MixedPair<A,B> {  
2     private A first;  
3     private B second;  
4  
5     public MixedPair(A first, B second) {  
6         this.first = first;  
7         this.second = second;  
8     }  
9     public A getFirst() {return first;}  
10    public B getSecond() {return second;}  
11  
12    public boolean sameType() {  
13        return first.getClass() == second.getClass();  
14    }  
15    public String toString() {  
16        return "[" + first + "," + second + "]";  
17    }  
18 }
```

Il metodo `getClass()`, ereditato da [Object](#), ritorna il nome della classe

VINCOLI SU TIPI GENERICI

- Un tipo generico può essere sostituito con qualsiasi tipo
- Spesso è invece utile imporre vincoli sui tipi generici
 - Evita la possibilità di incorrere in comportamenti indesiderati
 - Ad esempio, un algoritmo di ordinamento su un tipo generico non ha senso se lanciato su oggetti non confrontabili
- Java offre la possibilità di aggiungere vincoli ai tipi generici
 - Possiamo imporre che il tipo generico abbia qualche proprietà
 - Specifichiamo tali proprietà utilizzando la keyword **extends**
 - In pratica, imponiamo che il tipo generico sia una sottoclasse di una qualche altra classe
 - Utilizziamo la keyword **extends** anche se vogliamo imporre che il tipo generico *implementi* una interfaccia

ESEMPIO DI VINCOLO SU TIPI GENERICI

- Imponiamo che gli oggetti della classe *Pair* siano confrontabili

```
1 public class SortablePair<T extends Comparable<T>> {  
2     private T first, second;  
3  
4     public SortablePair(T first, T second) {  
5         this.first = first;  
6         this.second = second;  
7     }  
8     public T getFirst() {return first;}  
9     public T getSecond() {return second;}  
10  
11     public void sort() {  
12         if(first.compareTo(second) > 0) {  
13             T tmp = this.first;  
14             this.first = this.second;  
15             this.second = tmp;  
16         }  
17     }  
18     public String toString() {  
19         return "[" + first + "," + second + "];"  
20     }  
21 }
```

Possiamo invocare *compareTo()* (riga 12) perché *T* implementa [Comparable](#)

ARRAY DI TIPI GENERICI

- In Java la gestione di array di tipi generici è un po' complessa
 - Il tipo degli array è verificato a runtime

```
1 String[] a = new String[1];  
2 Object[] b = a;  
3 // Compilabile ma crash a runtime  
4 b[0] = Integer.valueOf(1);
```

- I tipi generici sono invece sostituiti durante la compilazione
- Non è pertanto possibile allocare un array generico nel seguente modo

```
1 public class GenericArray<T> {  
2     T[] array;  
3     public void GenericArray(int size) {  
4         array = new T[size]; // Errore  
5     }  
6 }
```

- Possiamo comunque aggirare questo problema in due modi

COME CREARE ARRAY DI TIPI GENERICI 1/2

- Possiamo fare typecasting su array di Object

```
1 public class GenericArray<T> {  
2     private T[] array;  
3     public GenericArray(int size) {  
4         @SuppressWarnings("unchecked")  
5         T[] tmp = (T[]) new Object[size];  
6         array = tmp;  
7     }  
8 }
```

Il compilatore lancia comunque un *warning*, che possiamo sopprimere

- Attenzione ai vincoli sul tipo generico

```
1 public class GenericArray<T extends Comparable<T>> {  
2     private T[] array;  
3     public GenericArray(int size) {  
4         @SuppressWarnings("unchecked")  
5         T[] tmp = (T[]) new Comparable[size];  
6         array = tmp;  
7     }  
8 }
```

Se allochiamo un array *Object[size]* il typecasting a *T[]* non è legale poiché il tipo *T* è *Comparable* a differenza del tipo *Object*

COME CREARE ARRAY DI TIPI GENERICI 2/2

- Possiamo lavorare direttamente con array di Object

```
1 public class GenericArray<T> {  
2     private Object[] array;  
3     public void genArray(int size) {  
4         array = new Object[size];  
5     }  
6 }
```

Possiamo assegnare qualsiasi oggetto ad un array di Object

- Necessario fare typecasting quando usiamo gli oggetti dell'array

```
1 public class GenericArray<T extends Comparable<T>> {  
2     private Object[] array;  
3     public T get(int i) {  
4         @SuppressWarnings("unchecked")  
5         T data = (T)array[i];  
6         return data;  
7     }  
8 }
```

- Gli array di tipi generici sono insicuri e dovrebbero essere solo privati

METODI STATICI E TIPI GENERICI

- I tipi generici non possono essere referenziati in un contesto statico

```
1 public class Sorting<T extends Comparable<T>> {  
2     // Errore  
3     public static void sort(T[] A) {...}  
4 }
```

Il metodo statico *sort* non conosce *T* prima che la classe venga istanziata

- Possiamo spostare la dichiarazione del tipo generico al metodo statico

```
1 public class Sorting {  
2     // OK  
3     public static <T extends Comparable<T>> void sort(T[] A) {...}  
4 }
```

- Possiamo o meno specificare il tipo al momento dell'invocazione

```
1 String A[] = {"topolino", "pippo", "pluto"};  
2  
3 Sorting.sort(A);           // OK  
4 Sorting.<String>sort(A);    // OK
```

INTERFACCIA ITERABLE E ITERATOR

- Una classe [Iterable](#) espone un metodo per allocare un iteratore

```
Iterator<T> iterator();
```

- Un [Iterator](#) è un oggetto che rappresenta una sorta di *cursore* con cui scandire una collezione di oggetti
- Un iteratore espone due metodi principali

```
boolean hasNext(); // true se c'è un altro elemento  
T next();          // ritorna l'elemento successivo
```

- Gli oggetti iterabili possono essere utilizzati in una struttura di controllo iterativa speciale indicata con il nome *for-each*
- Un iteratore è molto utile per scandire strutture dati concatenate

ESEMPIO DI SCANSIONE DI UNA LISTA SU ARRAY

```
1 // Struttura dati Lista su array
2 ArrayList<Integer> L = new ArrayList<>();
3 L.add(1); L.add(2); L.add(3);
4
5 // Scansione con iteratore (poiché ArrayList è Iterable)
6 int sum1 = 0;
7 Iterator<Integer> iter = L.iterator();
8 while(iter.hasNext())
9     sum1 = sum1 + iter.next();
10
11 // Scansione con for-each (poiché ArrayList è Iterable)
12 int sum2 = 0;
13 for(Integer n: L)
14     sum2 = sum2 + n;
15
16 // Scansione con indicizzazione
17 int sum3 = 0;
18 for(int i = 0; i < L.size(); i++)
19     sum3 = sum3 + L.get(i);
20
21 System.out.println(sum1 + " " + sum2 + " " + sum3);
```

Tre modi **equivalenti** per scorrere una struttura dati [ArrayList](#)

ESEMPIO DI SCANSIONE DI UNA LISTA CONCATENATA

```
1 // Struttura dati Lista concatenata
2 LinkedList<Integer> L = new LinkedList<>();
3 L.add(1); L.add(2); L.add(3);
4
5 // Scansione con iteratore (poiché LinkedList è Iterable)
6 int sum1 = 0;
7 Iterator<Integer> iter = L.iterator();
8 while(iter.hasNext())
9     sum1 = sum1 + iter.next();
10
11 // Scansione con for-each (poiché LinkedList è Iterable)
12 int sum2 = 0;
13 for(Integer n: L)
14     sum2 = sum2 + n;
15
16 // Scansione con indicizzazione COSTO?
17 int sum3 = 0;
18 for(int i = 0; i < L.size(); i++)
19     sum3 = sum3 + L.get(i);
20
21 System.out.println(sum1 + " " + sum2 + " " + sum3);
```

Tre modi **non equivalenti** per scorrere una struttura dati [LinkedList](#)

- Ridefinire l'interfaccia della struttura dati *Dictionary* con tipi generici
- Commentare i metodi ed il loro funzionamento
- Ricordiamo che la struttura dati Dizionario espone i metodi
 - SEARCH(Key k): cerca e ritorna l'oggetto associato alla chiave k
 - INSERT(Key k, Data d): inserisce la coppia (k,d) nel dizionario
 - DELETE(Key k): rimuove i dati associati alla chiave k

ESERCIZIO

- Aggiungere alla nostra implementazione della classe Sorting le versioni generiche degli algoritmi di ordinamento

```
1 public class Sorting {  
2  
3     public static <T extends Comparable<T>> void insertionsort(T A[]) {  
4         ..  
5     }  
6  
7     public static <T extends Comparable<T>> void selectionsort(T A[]) {  
8         ..  
9     }  
10  
11    public static <T extends Comparable<T>> void mergesort(T A[]) {  
12        ..  
13    }  
14  
15    public static <T extends Comparable<T>> void quicksort(T A[]) {  
16        ..  
17    }  
18    ...  
19 }
```

Nota: non possiamo usare questi metodi su array di int